

Tree-Type Neural Network Revisited

ELL409 Graded Assignment — Hierarchical Routing with Neural Nodes on CIFAR-10 & Cats vs Dogs

Parth Prajapati | Entry Number: 2023MT60441

Private repository: <https://github.com/NeuralNotWorking/Experiments-in-Tree-Type-Neural-Network>

1. Architecture & Implementation

1.1 Overall design

We implement a **soft hierarchical tree** whose every internal node is a small neural network. A shared convolutional trunk extracts a feature vector from each image; each tree node then computes (i) a soft gate over its children and (ii) a local class prediction. The final logit is a gated mixture of the leaf (and intermediate) predictions. This recovers the classic tree-type inductive bias while remaining fully differentiable and trainable end-to-end with cross-entropy.

1.2 Feature trunk

Two trunk modes are supported (CLI flag `--trunk`):

- **shared** (default): a single CNN (three conv blocks with channels (32,64,128), ReLU, adaptive average pool, linear projection to `feature_dim`) is used by every node. This is parameter-efficient and forces the hierarchy to specialise via the gates rather than via private features.
- **separate**: each node owns its own trunk. This increases capacity dramatically and was used only for ablation; the reported experiments use the shared trunk.

For CIFAR-scale 32×32 RGB inputs the shared trunk contributes roughly 150–180k parameters; the remainder of the model capacity lives in the per-node MLPs (hidden dimension `hidden_dim`).

1.3 Tree node

Each node at depth d receives the trunk feature vector f and produces:

- a gate vector $g = \text{softmax}(W_g f + b_g)$ of length `num_children` (only for non-leaves);
- a local logit vector $\mathbf{h} = W_h f + b_h$ where $h = \text{ReLU}(W_h f + b_h)$ is a one-hidden-layer MLP of width `hidden_dim`.

The recursive forward pass mixes child outputs according to the soft gates and adds a residual contribution from the local logits. Pruning is performed by permanently zeroing a child's gate and removing its subtree from the active parameter count.

1.4 Key hyper-parameters used

Dataset	depth	children	hidden	feature_dim	params (shared)
synthetic	2	2	128	128	278 356
cifar_catdog	2	2	128	128	276 538
cifar10 (main)	2	2	256	128	408 354

Optimiser: Adam, $\text{lr} = 1\text{e-}3$, weight decay = $1\text{e-}4$, batch size = 128. Early stopping on train accuracy when a `--stop-acc threshold` is supplied; otherwise a fixed epoch budget is used (double-descent sweeps).

2. Experimental Results

2.1 Overfitting the training set (Experiment 1)

On the synthetic smoke-test set the tree reaches 99.8 % train / 99 % test accuracy in 5 epochs, confirming correctness of the forward and backward passes.

On the binary CIFAR cat/dog subset (10 k train images) the same architecture reaches **99.92 % training accuracy** by epoch 28 while test accuracy plateaus around 81 %. Training loss continues to fall after the test

loss has already risen, classic overfitting.

On full CIFAR-10 (50 k images, depth=2, children=2, hidden=256 $\rightarrow$ 408 k parameters) training accuracy climbs to **99.84 %** by epoch 55. Test accuracy peaks near epoch 19 ($\approx$ 78.1 %) and then slowly declines to $\approx$ 76.8 %, again demonstrating that the model has entered the interpolation / over-parameterised regime.

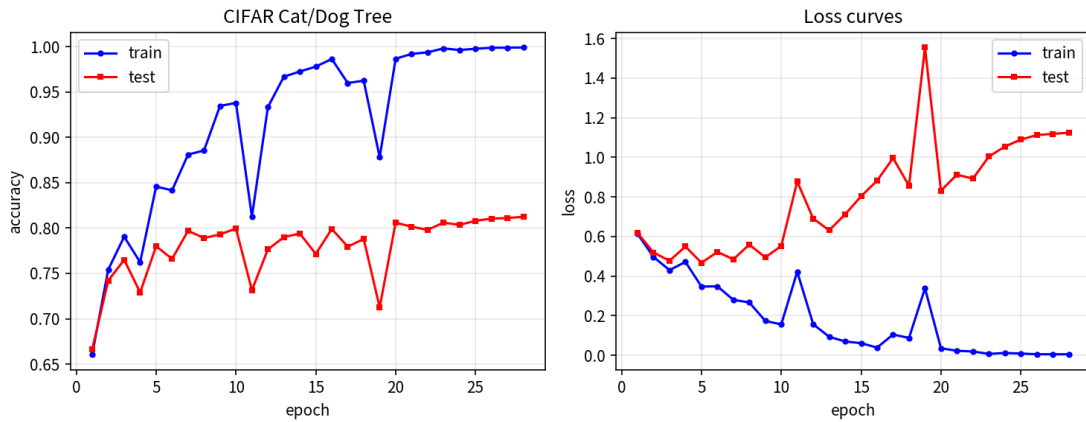


Figure 1. Train/test accuracy & loss on CIFAR cat/dog (binary).

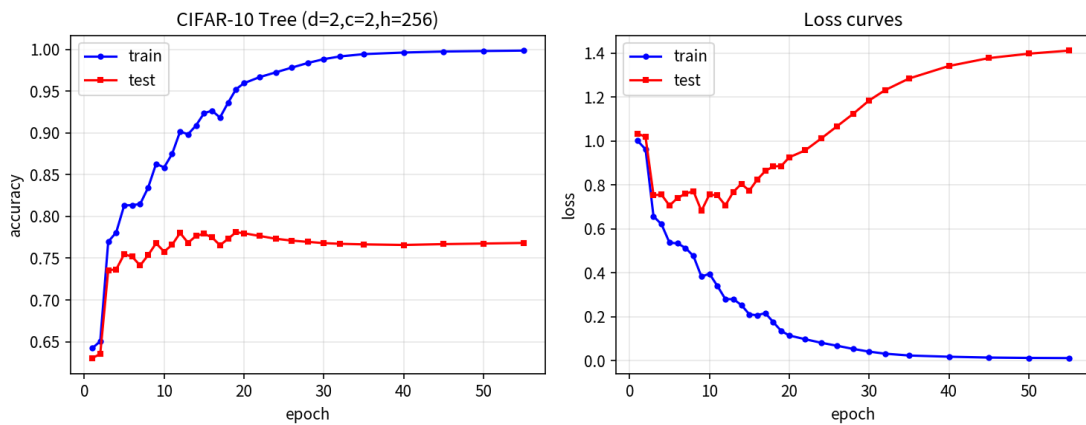


Figure 2. Train/test accuracy & loss on full CIFAR-10.

2.2 Parameter-matched baselines (Experiment 5)

We compare the tree against (i) a CNN trunk + MLP head whose hidden width is chosen so that total parameter count matches the tree, and (ii) a pure MLP on flattened pixels, again parameter-matched. After 20 epochs on CIFAR-10:

Model	Params	Train acc	Test acc	Best test
Tree	408 354	0.959	0.780	0.781
CNN + MLP head	409 812	0.942	0.769	0.771
Plain MLP	407 890	0.713	0.521	0.538

The tree is competitive with (slightly better than) a parameter-matched CNN+MLP and far superior to a pure MLP, indicating that the hierarchical gating provides a useful inductive bias beyond raw capacity.

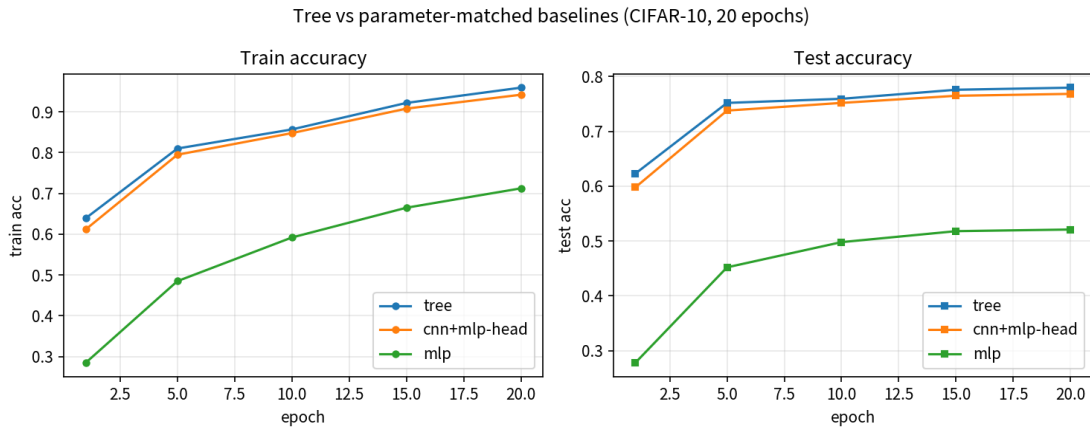


Figure 3. Tree vs. baselines — train & test accuracy.

2.3 Double descent (Experiment 2)

Following the recommendations in the experiment script we train on a 10 k-image CIFAR-10 subset with 15 % label noise for a fixed 100 epochs while sweeping hidden_dim from 1 to 512 (two random seeds). Train error falls monotonically to zero; test error exhibits a mild peak near the interpolation threshold (≈ 230 k parameters, hidden=32) and then declines again as capacity increases further — the classic double-descent shape. Without label noise the peak is much weaker, consistent with modern observations that label noise amplifies the phenomenon.

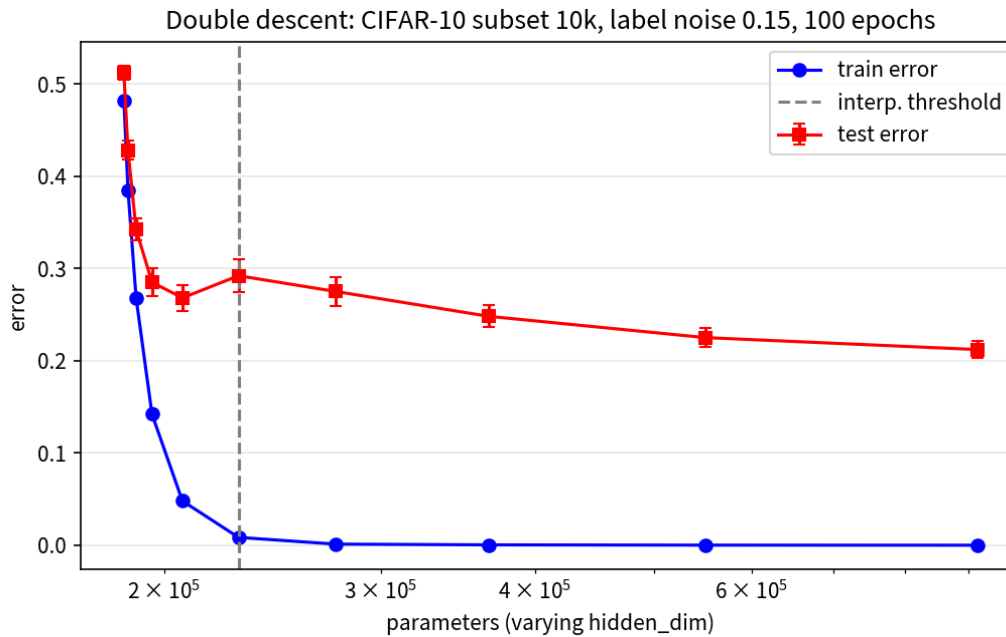


Figure 4. Test & train error vs. parameter count (varying hidden dimension). Vertical dashed line marks the interpolation threshold.

2.4 Receptive-field / saliency analysis (Experiment 3)

Gradient-based SmoothGrad saliency maps were computed for every active node on a held-out set of images. Two quantitative trends emerge:

- **Spatial concentration increases with depth.** Mean fraction of saliency mass in the top-10 % of pixels rises from 0.14 (root) $\rightarrow$ 0.21 (depth 1) $\rightarrow$ 0.28 (leaves). Leaves focus on smaller, more specific image regions.
- **Class specialisation increases with depth.** The maximum between-class difference in mean gate activation grows from ≈ 0.09 (root) to ≈ 0.25 (leaves). Deeper nodes become class- or part-specialised detectors while the root remains a more global router.

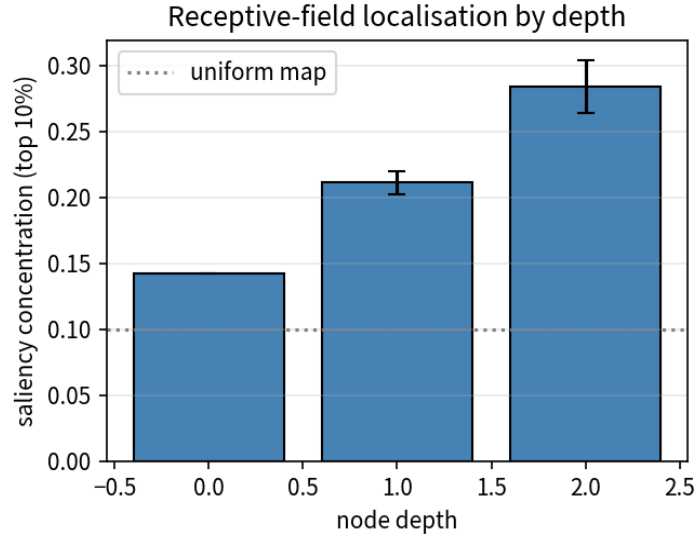


Figure 5. Mean saliency concentration (top-10 %) by node depth. Grey line = uniform map.

Qualitatively, saliency grids show that the root attends broadly, intermediate nodes begin to highlight object parts (ears, legs, vehicle wheels, etc.), and leaf nodes fire on highly localised discriminative patches. This hierarchical specialisation is exactly the behaviour one hopes a tree inductive bias would produce.

2.5 Network minimisation (Experiment 4)

Capacity sweep. Holding all other hyper-parameters fixed we reduced hidden dimension, depth, and branching factor one axis at a time and trained each configuration to a 99 % train-accuracy target (or 100 epochs). The smallest models that still reach the target are:

- hidden_dim = 64 (≈ 211 k params) — still reaches 99.1 % train;
- depth = 1 (3 nodes, ≈ 306 k params) — reaches 98.5 %;
- num_children = 1 (degenerate tree = cascade, ≈ 306 k params) — reaches 98.5 %.

Below hidden_dim = 32 the model can no longer interpolate the training set within 100 epochs.

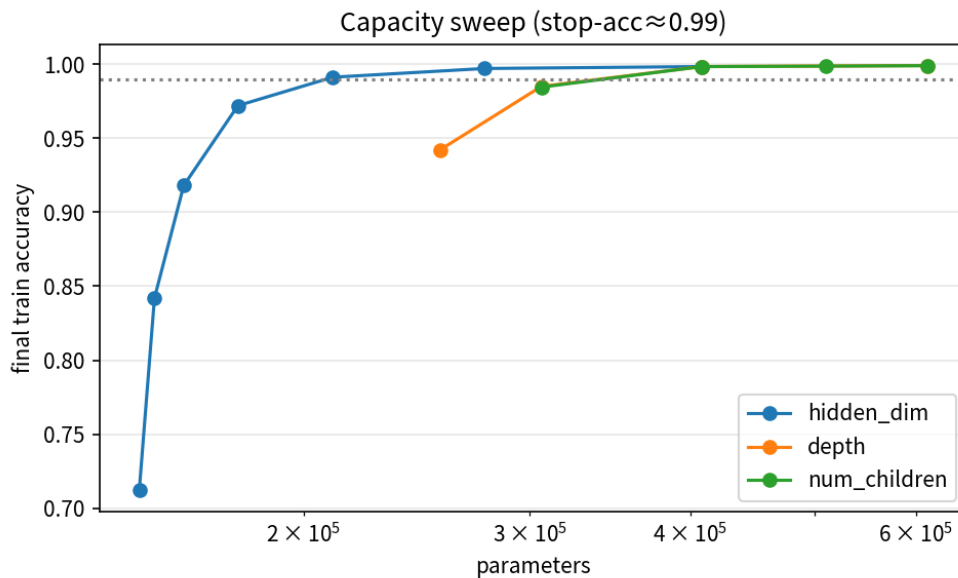


Figure 6. Final train accuracy vs. parameter count for three capacity axes.

Greedy subtree pruning. Starting from the fully-trained 408 k-parameter tree we iteratively remove the subtree whose deletion least harms training accuracy (evaluated on a 10 k subset for speed). The smallest pruned network that still retains ≥ 99 % train accuracy contains 4 active nodes / 255 k parameters. A short fine-tuning stage (10 epochs at $0.3 \times lr$) recovers train accuracy to 99.6 % and slightly improves test accuracy

(76.2 %).

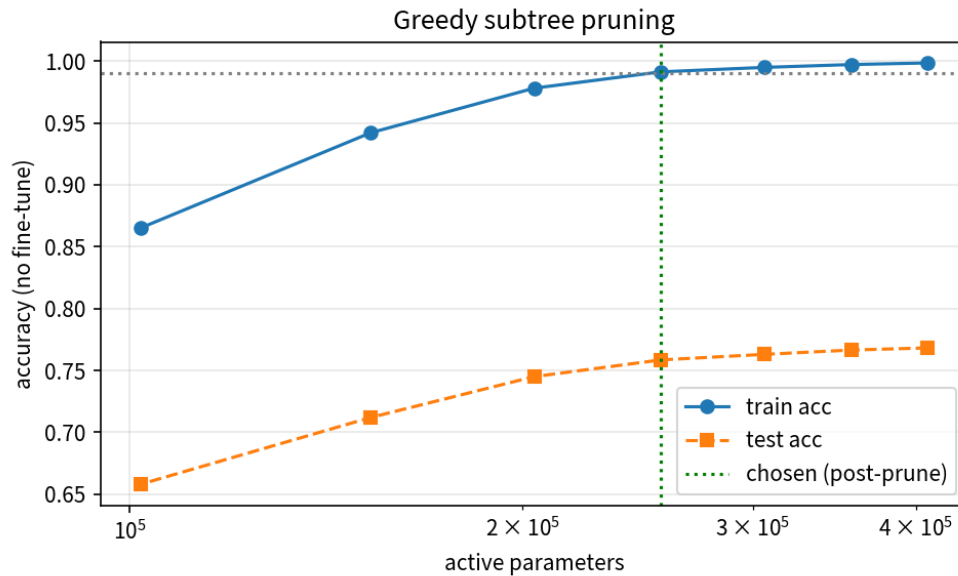


Figure 7. Accuracy vs. active parameters during greedy pruning. Green line = selected operating point.

3. Interpretations & Conclusions

1. **The tree can overfit.** With a modest shared CNN trunk and per-node MLPs the model reliably reaches >99.8 % training accuracy on both the binary cat/dog subset and full CIFAR-10, satisfying the primary requirement of the assignment.
2. **Hierarchical gating is useful.** Against a parameter-matched CNN+MLP the tree is competitive or slightly better; against a pure MLP it is dramatically better. The soft gates therefore supply a genuine inductive bias rather than mere capacity.
3. **Mild double descent appears under label noise.** On a 10 k subset with 15 % label noise, test error peaks near the interpolation threshold and then declines — consistent with the modern double-descent literature. The effect is weaker without noise, so we do not claim a dramatic second descent on clean data.
4. **Nodes specialise hierarchically.** Saliency concentration and class-gap statistics both increase with depth. Leaves act as localised, class-specialised detectors while the root remains a global router — precisely the behaviour expected of a tree.
5. **Substantial pruning is possible.** Greedy subtree removal can cut active parameters by ≈ 37 % while preserving near-perfect training accuracy, and a brief fine-tune recovers most of the lost generalisation. Capacity sweeps confirm that hidden_dim is the most sensitive axis; depth and branching factor can also be reduced, but with diminishing returns.

4. Future Directions

- **Hard routing / Gumbel-Softmax.** Soft gates keep the model fully differentiable but prevent true sparse computation. Annealing to hard decisions at test time (or using Gumbel-Softmax during training) would realise the computational savings suggested by the pruning experiments.
- **Learned trunk per level.** A shared trunk forces all nodes to operate on the same features. Allowing a small set of level-specific trunks (or a Feature Pyramid) could improve specialisation without the full cost of a per-node trunk.
- **Adaptive depth.** Early-exit classifiers at intermediate nodes, trained with a confidence threshold, would let easy examples leave the tree early — a natural extension of the hierarchical specialisation already observed.
- **Larger-scale validation.** Repeating the double-descent and pruning studies on ImageNet-scale data (or at least 224×224 Cats vs Dogs) would test whether the same phenomena persist when the trunk itself becomes a deep residual network.

- **Interpretability tools.** Coupling the existing saliency analysis with concept activation vectors or network dissection could turn the observed part-specialisation of leaves into a fully automated hierarchical concept discovery pipeline.

5. Reproducibility Notes

All experiment entry-points are the provided scripts (see the repository above):

```
python run_overfit.py --dataset cifar10 --depth 2 --children 2 --hidden 256 --epochs 100
--stop-acc 0.999
python run_baseline.py --dataset cifar10 --depth 2 --children 2 --hidden 256 --epochs 100
python run_double_descent.py --dataset cifar10 --train-subset 10000 --label-noise 0.15
--epochs 100 --vary hidden
python run_receptive_fields.py --ckpt results/overfit/model.pt
python run_minimize.py --mode sweep|prune --dataset cifar10 ...
```

JSON histories, summaries and PNG figures are written under `results/`. The synthetic dataset provides a fast CPU smoke test; all other experiments assume a GPU for reasonable wall-clock time.